

Lecture Notes:

The Transformer Architecture

GCC1734 - Inteligência Artificial / CIC1205 - Machine Learning

Eduardo Bezerra

June 17, 2026

0 Contents

Before You Read	3
Learning Objectives	3
1 Introduction: Why Replace Recurrence?	4
1.1 Recap: Two Bottlenecks of RNNs	4
1.2 The Core Idea: Attention as Direct Lookup	4
2 Self-Attention: Queries, Keys, and Values	7
2.1 From Embeddings to Q, K, V	7
2.2 The Attention Formula	8
2.3 A Worked Example	9
3 Scaling, Multi-Head Attention, and Self-Attention as a Layer	11
3.1 Why Divide by $\sqrt{d_k}$?	11
3.2 Multi-Head Attention	12
3.3 Self-Attention as a Layer: Shapes and Properties	13
4 Positional Encoding	15
4.1 The Problem, Restated	15
4.2 The Sinusoidal Formula	15
4.3 A Worked Example: $d_{\text{model}} = 4$	15
4.4 Two Key Properties	16
4.5 Learned vs. Fixed Positional Encodings	17
5 Residual Connections and Layer Normalization	19
5.1 Motivation: Stacking Many Layers	19
5.2 Residual Connections	19
5.3 Layer Normalization	20
5.4 Putting Them Together: The Transformer Sub-Layer Pattern	21
6 Assembling the Architecture: Encoder, Decoder, and Cross-Attention	23
6.1 The Encoder Block	23
6.2 The Decoder Block: Masked Self-Attention	23
6.3 Two Types of Masks: A Quick Clarification	24
6.4 Cross-Attention: Connecting Encoder and Decoder	25
6.5 The Full Encoder-Decoder Transformer	26
7 Transformer Families: Encoder-Only, Decoder-Only, Encoder-Decoder	28
7.1 Not All Transformers Are Encoder-Decoder	28
7.2 Architectural Differences in Detail	28
8 Implementation in PyTorch	31
8.1 Two Types of Masks: PyTorch Conventions	31
8.2 PyTorch Transformer Modules	31
8.3 A Minimal Self-Attention Layer from Scratch	31
8.4 Transformer Encoder for Text Classification	32

8.5	Building a Causal Language Model (Decoder-Only)	34
9	Computational Complexity and Efficient Attention	36
9.1	The $O(n^2)$ Bottleneck	36
9.2	Efficient Attention Variants	36
10	Common Pitfalls, Debugging, and Final Remarks	39
10.1	Pitfalls Specific to Transformers	39
10.2	The “Three Lines That Destroy a Transformer”	39
10.3	Summary of Key Concepts	39
10.4	Recommended Next Steps	40

0 Before You Read

These notes build directly upon the RNN/LSTM/GRU lecture notes. The following concepts are assumed to be already understood. Review them if necessary before proceeding.

- The hidden state $\mathbf{h}_t$ as a fixed-size, lossy summary of a sequence's past.
- The vanishing/exploding gradient problems and why they limit how far information can travel through a recurrent network.
- The sequential (one-step-at-a-time) nature of RNN computation, and why this prevents parallelization across time steps.
- Basic linear algebra: matrix multiplication, dot products, the softmax function.
- The general structure of a supervised training loop: forward pass, loss computation, backward pass, optimizer step.
- Word embeddings as dense vector representations of tokens (Section 7.2 of the embedding notes).

0 Learning Objectives

After studying these notes, you should be able to:

- explain the two fundamental limitations of RNNs (sequential computation and the long-range information bottleneck) and how they motivate the Transformer;
- describe the self-attention mechanism, derive the query/key/value formulation, and compute attention weights and outputs for a small example by hand;
- explain why attention scores are scaled by $\sqrt{d_k}$ and why this matters for training stability;
- describe multi-head attention and explain what additional expressive power it provides over single-head attention;
- explain why positional encoding is necessary, derive the sinusoidal positional encoding formula, and explain its extrapolation property;
- describe the role of residual connections and layer normalization in stabilizing deep Transformer training;
- describe the encoder-decoder Transformer architecture, including the role of masked self-attention and cross-attention;
- distinguish encoder-only, decoder-only, and encoder-decoder Transformer families and map them to representative models (BERT, GPT, T5) and tasks;
- implement a minimal self-attention layer and a Transformer encoder block in PyTorch using `nn.MultiheadAttention` and `nn.TransformerEncoderLayer`;
- analyze the computational complexity of self-attention with respect to sequence length and explain why this motivates efficient-attention research.

1 Introduction: Why Replace Recurrence?

1.1 Recap: Two Bottlenecks of RNNs

The RNN-based lecture notes ended with two structural limitations of recurrent architectures. It is worth restating them precisely, because the entire Transformer architecture can be understood as a direct response to these two problems.

The Two Bottlenecks

1. **Sequential computation bottleneck.** In an RNN, $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, which depends on $\mathbf{h}_{t-2}$, and so on. This chain of dependencies means the hidden states *must* be computed one at a time, in order. Even with a massive GPU capable of performing millions of operations in parallel, an RNN cannot exploit this parallelism across the time dimension during training: step t simply cannot start before step $t - 1$ finishes.
2. **Long-range information bottleneck.** Even with the gating mechanisms of the LSTM and GRU, information about token 1 that needs to influence the prediction at token 1000 must pass through 999 intermediate hidden states, each of which has a *fixed size*. The hidden state is a compressed summary; it cannot grow to accommodate longer sequences. Relevant information can be diluted or overwritten along the way.

Bridge from RNNs

In the RNN notes, the hidden state was described as “a running notepad of limited size.” The Transformer’s central design choice is to *stop using a notepad altogether*: instead of compressing the past into a single vector and carrying it forward step by step, the Transformer keeps *all* token representations simultaneously available within the context window, and lets each token directly query every other token whenever it needs information from them. There is no sequential relay race to slow down computation, and no fixed-size summary that must carry the entire past.

A precise qualification is worth stating up front: the Transformer does *not* eliminate all information bottlenecks. Each token is still represented by a fixed-dimensional vector of size d_{model} , and the mixing via attention is still a weighted combination of value vectors. What the Transformer eliminates is the specifically *recurrent* bottleneck, in which all past information must be funneled through a single hidden state that is rewritten at every step. Long-range dependencies that would require surviving hundreds of recurrent state updates can now be expressed as a single direct attention connection.

1.2 The Core Idea: Attention as Direct Lookup

Imagine you are reading the sentence:

“The animal didn’t cross the street because it was too tired.”

To resolve what *it* refers to, an RNN would need the information about *animal* (token 2) to survive, encoded inside the hidden state, all the way to token 10 (*it*) — a journey of 8 recurrent steps, each one a potential source of dilution.

Self-attention takes a radically different approach: when processing the word *it*, the model computes a *direct* compatibility score between *it* and *every other word in the sentence*, including *animal* and *street*. If the score for *animal* is high, the representation of *it* is updated to directly incorporate information from *animal* — in a single step, regardless of the distance between the two words.

Self-Attention, Informally

Self-attention is a mechanism that, for each token in a sequence, computes a weighted combination of the representations of *all* tokens in the sequence (including itself), where the weights reflect how relevant each token is to the one being updated. The weights are not fixed: they are computed dynamically from the content of the tokens themselves, and they are learned end-to-end via backpropagation.

The Transformer Storyline

- Step 1.** *The Problem.* RNNs cannot parallelize across time and suffer an information bottleneck over long sequences. (Section 1)
- Step 2.** *The Core Mechanism: Self-Attention.* Every token directly attends to every other token via Query, Key, and Value projections. (Section 2)
- Step 3.** *Scaling and Stabilizing.* Why attention scores are divided by $\sqrt{d_k}$, and why a single attention head is often too restrictive. (Section 3)
- Step 4.** *Restoring Order: Positional Encoding.* Self-attention is permutation-equivariant by construction; positional encodings inject order back in. (Section 4)
- Step 5.** *Making It Trainable at Depth: Residuals and LayerNorm.* Skip connections and normalization let Transformers stack dozens of layers. (Section 5)
- Step 6.** *Assembling the Architecture.* Encoder, decoder, masked self-attention, cross-attention, and the full encoder-decoder Transformer. (Section 6)
- Step 7.** *The Landscape.* Encoder-only vs. decoder-only vs. encoder-decoder families; implementation in PyTorch; complexity and efficient attention. (Sections 7–9)

Entry Ticket 1

Before reading further, run the following PyTorch snippet and bring your observations to class:

```

1 import torch
2 import torch.nn as nn
3
4 # A single multi-head self-attention layer
5 mha = nn.MultiheadAttention(embed_dim=16, num_heads=4,
6                               batch_first=True)
7 x = torch.randn(2, 5, 16) # (batch, seq_len, embed_dim)
8
9 out, attn_weights = mha(x, x, x) # self-attention: Q=K=V=x
10 print(f"Output shape: {out.shape}")

```

```
10 print(f"Attention weights shape: {attn_weights.shape}")
```

1. Why are `x` passed three times to `mha`? What do these three arguments represent conceptually?
2. The output shape matches the input shape exactly. Why is this property essential for stacking multiple Transformer layers?
3. `attn_weights` has shape (2, 5, 5). What does each of the 5x5 matrices represent? What would you expect the row sums to be, and why?
4. Reread the RNN notes' description of the hidden state as "a fixed-size, lossy summary." Does self-attention have an analogous fixed-size bottleneck? Why or why not?

2 Self-Attention: Queries, Keys, and Values

2.1 From Embeddings to Q, K, V

Self-attention operates on a sequence of embedding vectors. Suppose the input sequence has n tokens, each represented by a d -dimensional embedding (these embeddings already encode token identity, but *not yet* position — that comes in Section 4). Stack these embeddings as rows of a matrix:

$$X \in \mathbb{R}^{n \times d}.$$

The model learns three weight matrices, $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$, and uses them to produce three projections of X :

Query, Key, Value Projections

$$Q = X\mathbf{W}_Q \in \mathbb{R}^{n \times d_k}, \quad K = X\mathbf{W}_K \in \mathbb{R}^{n \times d_k}, \quad V = X\mathbf{W}_V \in \mathbb{R}^{n \times d_v}.$$

Each row $Q_i \in \mathbb{R}^{d_k}$, $K_i \in \mathbb{R}^{d_k}$, $V_i \in \mathbb{R}^{d_v}$ is associated with token i . In the standard Transformer, $d_v = d_k$ for simplicity, but the two dimensions serve different roles and need not be equal in general.

These three names come from an information-retrieval analogy that is worth taking seriously, because it makes the computation in Section 2.2 much easier to remember.

The Query/Key/Value Analogy

Think of a library catalog system.

- A **query** (Q_i) is the question that token i asks: “*what kind of information am I looking for?*”
- A **key** (K_j) is a label attached to token j describing *what information it offers*: “*here is what I have to provide.*”
- A **value** (V_j) is the actual content that token j will hand over *if* its key matches a query well.

Token i compares its query Q_i against *every* key $K_1, \dots, K_n$ (including its own key K_i). The comparison scores determine how much of each value $V_1, \dots, V_n$ gets mixed into the new representation of token i . Crucially, Q , K , and V are three *different linear projections of the same input X* : every token simultaneously plays the role of an asker (via its query), an advertiser (via its key), and a content provider (via its value).

Myth: Q, K, and V Are Different Tokens

A common confusion is to think that queries come from one set of tokens, keys from another, and values from a third. In *self*-attention, Q , K , and V are three projections of the *same* sequence X . Every token i has its own Q_i , K_i , and V_i , all derived from the same embedding X_i via different weight matrices. The distinction is functional (what role each vector plays in the computation), not about which tokens are involved. This will change in Section 6, where *cross*-attention uses Q from one sequence and K, V from another.

2.2 The Attention Formula

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (1)$$

The softmax is applied *row-wise*: each row of $QK^\top/\sqrt{d_k}$ is normalized independently so that its entries are non-negative and sum to 1.

It is worth unpacking Eq. 1 term by term, since each piece corresponds to a distinct conceptual step.

Step 1: Compatibility scores. The matrix $S = QK^\top \in \mathbb{R}^{n \times n}$ has entries $s_{ij} = Q_i \cdot K_j$, the dot product between the query of token i and the key of token j . A large dot product means Q_i and K_j point in similar directions, i.e., token j offers something token i is looking for.

Step 2: Scaling by $\sqrt{d_k}$. The raw dot products are divided by $\sqrt{d_k}$ before the softmax. Section 3.1 explains in detail why this is necessary; for now, note that without this step, the *spread* among the scores can become very large for large d_k , causing softmax to saturate and gradients to vanish.

Step 3: Row-wise softmax.

$$\alpha_{ij} = \frac{\exp(s_{ij}/\sqrt{d_k})}{\sum_{j'=1}^n \exp(s_{ij'}/\sqrt{d_k})}.$$

The resulting matrix $\alpha \in \mathbb{R}^{n \times n}$ contains the *attention weights*. Row i of α is a probability distribution over all n tokens, describing how token i distributes its “attention” across the sequence (including itself).

Step 4: Weighted sum of values.

$$Z = \alpha V \in \mathbb{R}^{n \times d_v}, \quad Z_i = \sum_{j=1}^n \alpha_{ij} V_j.$$

The output representation Z_i for token i is a convex combination of *all* value vectors $V_1, \dots, V_n$, weighted by how relevant the model judged each token j to be for token i .

Self-Attention Is a Differentiable, Content-Based Lookup Table

Eq. 1 can be read as a soft, differentiable version of a dictionary lookup: Q_i is the lookup key you provide, K_j are the keys stored in the dictionary, and V_j are the corresponding stored values. Instead of retrieving a single exact match, you retrieve a weighted blend of *all* values, where the weights depend on how well your query matches each stored key. Because every operation involved (dot products, softmax, weighted sum) is differentiable, the entire retrieval process — including *what to look for* ($\mathbf{W}_Q$), *what to advertise* ($\mathbf{W}_K$), and *what to retrieve* ($\mathbf{W}_V$) — is learned by

gradient descent.

What Self-Attention Does *Not* Do

Self-attention is a powerful mechanism, but it is easy to over-attribute capabilities to it. Four important limitations:

- **It is not persistent memory.** Self-attention does not store information between different input sequences. It recomputes contextualized representations from the current input every time. There is no cross-sequence “memory” analogous to the hidden state at the boundary between RNN sequences.
- **It has no notion of order.** As established above (and central to Section 4), a self-attention layer with no positional information is entirely order-blind.
- **Attention weights are not a complete explanation of the model’s decisions.** When one token attends strongly to another, this does not fully explain *why* the model makes a prediction: the value vectors V_j and the downstream FFN layers also determine the final output. Interpreting attention weights as causal explanations is an active and contested research area.
- **Each token is still represented by a fixed-dimensional vector.** Self-attention eliminates the recurrent bottleneck but each token representation still has fixed size d_{model} . The Transformer’s advantage is that information can flow *directly* between any two token representations in a single layer, without passing through intermediate states.

2.3 A Worked Example

Consider the sentence “*O gato comeu o peixe*” (“The cat ate the fish”), with $n = 5$ tokens and embedding dimension $d = 3$. Suppose the embedding matrix is:

$$X = \begin{bmatrix} 0.2 & 0.1 & 0.0 \\ 0.8 & 0.5 & 0.1 \\ 0.7 & 0.9 & 0.6 \\ 0.3 & 0.1 & 0.2 \\ 0.9 & 0.7 & 0.8 \end{bmatrix} \begin{array}{l} \leftarrow \text{“O”} \\ \leftarrow \text{“gato”} \\ \leftarrow \text{“comeu”} \\ \leftarrow \text{“o”} \\ \leftarrow \text{“peixe”} \end{array}$$

After learning projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{3 \times 4}$ (so $d_k = d_v = 4$), the model produces $Q, K, V \in \mathbb{R}^{5 \times 4}$, one row per token. A plausible (illustrative) attention weight distribution for row 3 (the query token “*comeu*”/“*ate*”) after applying Eq. 1 is:

$$\alpha_{3,\cdot} = [0.05, 0.35, 0.10, 0.05, 0.45].$$

This says: when updating the representation of “*comeu*”, the model assigns 35% of its attention to “*gato*” (the subject) and 45% to “*peixe*” (the object). The new representation of “*comeu*” is:

$$Z_3 = \sum_{j=1}^5 \alpha_{3j} V_j = 0.05V_1 + 0.35V_2 + 0.10V_3 + 0.05V_4 + 0.45V_5,$$

a vector that now encodes not just the verb in isolation, but “*ate*” in the specific context of a cat eating a fish.

How Do These Weights Get Learned? And What Do They Really Mean?

Nothing in the formula tells the model that verbs should attend to their subjects and objects. At initialization, $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are random; in expectation, by symmetry, no token is systematically preferred over any other, so attention patterns are essentially unstructured at the start. During training — typically on a task like predicting the next token — backpropagation adjusts these matrices so that attention patterns that reduce prediction error are reinforced.

A word of caution on interpretation: the example above might suggest that attention weights directly reveal the model’s “reasoning,” e.g., that the model “understands” subject-verb-object structure. Attention weights can sometimes correlate with linguistic relations, but they should not be treated as a transparent window into the model’s internal logic. The value vectors V_j and subsequent FFN layers also shape the output; a high attention weight to a token does not straightforwardly mean that token “caused” the output.

Entry Ticket 2

Work through the following before reading about scaling and multi-head attention.

1. Suppose $Q_i = K_i$ for all i (the query and key projections are identical). What would the diagonal entries α_{ii} of the attention matrix tend to look like compared to the off-diagonal entries? Is this necessarily a problem?
2. In the worked example, $\alpha_{3, \cdot}$ sums to 1. Verify this using the given numbers. What operation in Eq. 1 guarantees this property?
3. If $n = 1000$ (a 1000-token document), what is the shape of the attention matrix α ? How does the *memory* required to store α scale as n doubles?
4. Compare the worked example here to the entry-ticket exercise from the RNN notes (computing $\mathbf{h}_2$ from $\mathbf{h}_1$ and $\mathbf{x}_2$). In the RNN, how many previous tokens directly influence $\mathbf{h}_2$? In self-attention, how many tokens directly influence Z_3 ?

3 Scaling, Multi-Head Attention, and Self-Attention as a Layer

3.1 Why Divide by $\sqrt{d_k}$?

The scaling factor $\sqrt{d_k}$ in Eq. 1 is easy to overlook, but it addresses a real and severe training problem.

Assume the entries of Q_i and K_j are independent random variables with mean 0 and variance 1 (a reasonable assumption near initialization, when weights are randomly initialized and inputs are normalized). Then the dot product $Q_i \cdot K_j = \sum_{l=1}^{d_k} Q_{il}K_{jl}$ is a sum of d_k independent terms, each with mean 0 and variance 1. By standard properties of variance for sums of independent variables,

$$\text{Var}(Q_i \cdot K_j) = \sum_{l=1}^{d_k} \text{Var}(Q_{il}K_{jl}) = d_k.$$

So the *standard deviation* of the raw dot product grows as $\sqrt{d_k}$. As d_k increases, the typical *spread* among scores s_{ij} grows, meaning the differences between the largest and smallest scores in a row become much larger.

Myth: Scaling Is Just a Minor Numerical Detail

It is tempting to dismiss the $\sqrt{d_k}$ factor as an arbitrary normalization constant. But its absence has a concrete, severe consequence: **softmax saturation**. To understand this, recall that softmax is invariant to adding a constant to all its inputs; what matters is not the absolute magnitude of the scores, but the *spread* among them. The softmax function $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$ has the property that as the spread among the logits grows large — the largest entry dwarfs the others — the output approaches a one-hot vector: almost all the probability mass concentrates on the single largest entry. In this saturated regime, the *gradient* of the softmax with respect to its inputs becomes vanishingly small almost everywhere.

This means: **without scaling, large d_k causes large spread among dot products, which causes softmax saturation, which causes vanishing gradients through the attention mechanism itself** — a new flavor of the vanishing gradient problem familiar from the RNN notes, but arising from a completely different mechanism (softmax saturation rather than repeated Jacobian multiplication along a chain).

Dividing by $\sqrt{d_k}$ rescales the dot products so that, under the same independence assumptions, $\text{Var}(s_{ij}/\sqrt{d_k}) = 1$, regardless of d_k . This keeps the spread of softmax inputs in a range where gradients remain informative, which is essential for training deep stacks of attention layers.

Bridge from RNNs

The RNN notes discussed gradient clipping as a *reactive* fix for exploding gradients — the problem is allowed to occur during a forward/backward pass and is then rescaled afterward. The $\sqrt{d_k}$ scaling in attention is a *proactive, architectural*

fix: it is built into the forward computation itself, ensuring the problematic regime (softmax saturation) is unlikely to be entered in the first place. This is a recurring theme in this chapter: where RNNs often relied on training-time interventions (clipping, careful initialization, curriculum schedules), the Transformer bakes stability directly into the architecture — scaling here, and residual connections plus layer normalization in Section 5.

3.2 Multi-Head Attention

A single attention computation, as described in Section 2, produces *one* attention matrix $\alpha \in \mathbb{R}^{n \times n}$ per layer. This means each token can express only *one* pattern of “what I’m looking for” via its single query vector Q_i . But language is full of *simultaneous*, independent relationships: a verb might need to attend to its subject *and* its object *and* a nearby adverb, all for different reasons. A single α matrix, computed from a single set of projections, must compromise between all of these needs, making a single attention head often too restrictive.

Multi-Head Attention

Instead of one attention computation with $d_k = d_{\text{model}}$, multi-head attention runs h *independent* attention computations (“heads”) in parallel, each operating on a smaller dimension $d_k = d_{\text{model}}/h$:

$$\text{head}_i = \text{Attention}\left(X\mathbf{W}_i^Q, X\mathbf{W}_i^K, X\mathbf{W}_i^V\right), \quad i = 1, \dots, h, \quad (2)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (3)$$

where each $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are *separate, learned* projection matrices for head i , and $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ projects the concatenated heads back to the model dimension.

Parameter Count Is (Roughly) Unchanged

A common point of confusion: does using h heads multiply the parameter count by h ? **No.** Because each head operates on a reduced dimension $d_k = d_{\text{model}}/h$, the combined size of all h matrices $\mathbf{W}_i^Q$ equals that of a single matrix $\mathbf{W}^Q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ — and the same holds for K and V . The output projection $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ is an additional parameter relative to a bare single-head layer, but of the same order of magnitude. Multi-head attention does not primarily add capacity in the sense of *more parameters*; it restructures the *same* parameter budget into h independent subspaces, each free to specialize in a different type of relationship. This analysis assumes $d_k = d_v = d_{\text{model}}/h$, which is the standard Transformer setting.

What Multiple Heads Buy You

Empirically, different heads in a trained Transformer specialize in different types of linguistic relationships: some heads attend primarily to adjacent tokens (local

syntax, e.g., article to noun), others to long-range dependencies (e.g., a pronoun attending back to its antecedent), and others to positional patterns (e.g., a head that always attends to the previous token, functioning somewhat like a learned bigram detector). Multi-head attention gives the model h independent “slots” in which such specialized patterns can emerge, analogous to how different filters in a CNN layer learn to detect different visual patterns at the same spatial location.

3.3 Self-Attention as a Layer: Shapes and Properties

It is useful to step back and treat self-attention (single- or multi-head) as a generic *layer* with a well-defined input/output contract, just as the RNN notes treated `nn.LSTM` as a layer.

- **Input:** $X \in \mathbb{R}^{n \times d_{\text{model}}}$, a sequence of n token representations.
- **Output:** $Z \in \mathbb{R}^{n \times d_{\text{model}}}$, a sequence of the *same* length and dimensionality, where each Z_i is a context-aware version of X_i .
- **Parameters:** independent of n . The projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}^O$ have fixed shapes regardless of sequence length, just as the RNN’s recurrent weight matrices did.
- **Permutation equivariance.** If you permute the rows of X (shuffle the input token order) and feed the result through self-attention, the output rows are permuted in exactly the same way — but otherwise unchanged. Self-attention, as defined so far, has **no notion of token order whatsoever**.

Myth: A Self-Attention Layer “Knows” the Sequence Order

This last property deserves special emphasis, because it is the single most common point of confusion for students new to Transformers, and it sets up the entire motivation for Section 4.

Formally, self-attention is **permutation-equivariant**: if you permute the n rows of the input X , the output Z rows are permuted in exactly the same way. “Equivariant” means *the output transforms consistently with the input*, not that the output is unaffected. (Contrast with *invariance*, where the output would be identical regardless of input permutation — that is a stronger condition not satisfied here.) The practical consequence is important: since the output for token i changes predictably when that token is reordered, but its relationship to the other output vectors is preserved, the model cannot extract any information about the original positions from the attention computation alone.

To make this concrete: the sentences “*O gato comeu o peixe*” and “*O peixe comeu o gato*” share the same set of word embeddings (just reordered). Without positional information, the *set* of output representations $\{Z_1, \dots, Z_5\}$ would be the same set for both sentences (just permuted), and the model would have **no way to distinguish who ate whom**. Self-attention operates on a “bag of tokens with rich pairwise interactions” — it is the positional encoding (Section 4) that turns this bag back into a *sequence*.

Entry Ticket 3

Reflect on scaling and multi-head attention before reading about positional encoding.

1. A model has $d_{\text{model}} = 512$ and uses $h = 8$ heads. What is d_k for each head? If instead $h = 1$ (single-head attention with $d_k = 512$), by what factor would the typical *standard deviation* of $QK^\top$ entries increase *before* scaling, compared to a head with $d_k = 64$?
2. Without the $\sqrt{d_k}$ scaling, would you expect training to fail immediately, or only become problematic as d_{model} (and hence d_k) increases? Justify your answer using the variance argument from Section 3.1.
3. Suppose someone proposes: “Let’s just make the model bigger by increasing h while keeping d_k fixed (so $d_{\text{model}} = h \cdot d_k$ grows with h).” Does this increase the parameter count of the attention layer? Does it increase the size of the attention matrix α computed by *each* head?
4. Two sentences, “O gato comeu o peixe” and “O peixe comeu o gato”, are passed through a self-attention layer with no positional information. Will the *set* of output vectors $\{Z_1, \dots, Z_5\}$ differ between the two sentences? Will the model be able to tell, from these outputs alone, which animal ate which?

4 Positional Encoding

4.1 The Problem, Restated

Section 3.3 established that self-attention is permutation-equivariant: it treats the input as an unordered set of token representations. Yet word order is essential to meaning. The Transformer’s solution is conceptually simple: *inject* information about each token’s position directly into its embedding, *before* the embedding is passed through any attention layer. Once positional information is baked into the representations, it participates in the Q , K , V projections and in all subsequent computations — the model does not receive a separate “position” input at any later stage. Position is entangled with token identity inside the representation itself from the very first layer onward.

Positional Encoding

For each position $\text{pos} \in \{0, 1, \dots, n-1\}$ in the input sequence, define a vector $P_{\text{pos}} \in \mathbb{R}^{d_{\text{model}}}$. The input to the first Transformer layer is then

$$Z_{\text{pos}} = X_{\text{pos}} + P_{\text{pos}},$$

where X_{pos} is the token embedding at position pos . The resulting vector Z_{pos} carries information about *both* the token’s identity and its position.

4.2 The Sinusoidal Formula

Vaswani et al. [2017] proposed a deterministic positional encoding based on sine and cosine functions of different frequencies:

Sinusoidal Positional Encoding

$$P_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad (4)$$

$$P_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad (5)$$

where pos is the position index, and i indexes pairs of dimensions ($i = 0, 1, \dots, d_{\text{model}}/2 - 1$).

It is worth working through a concrete, small example to build intuition before discussing the general properties.

4.3 A Worked Example: $d_{\text{model}} = 4$

Let $d_{\text{model}} = 4$, so there are two pairs of dimensions: (P_0, P_1) for $i = 0$ and (P_2, P_3) for $i = 1$.

Step 1: Compute the frequency denominators. The term $10000^{2i/d_{\text{model}}}$ controls how quickly each dimension oscillates as pos increases. For $d_{\text{model}} = 4$:

$$i = 0 : \quad 10000^{0/4} = 10000^0 = 1, \quad i = 1 : \quad 10000^{2/4} = 10000^{0.5} = 100.$$

Step 2: Interpret the denominators. A larger denominator means the argument of $\sin/\cos$ grows *more slowly* with pos , i.e., the wave oscillates more slowly.

- Dimensions (P_0, P_1) use denominator 1: $\sin(\text{pos})$ and $\cos(\text{pos})$ change substantially from one position to the next. These dimensions encode **local, fine-grained** position information.
- Dimensions (P_2, P_3) use denominator 100: $\sin(\text{pos}/100)$ and $\cos(\text{pos}/100)$ change very slowly. These dimensions encode **coarse, long-range** position information.

Step 3: Tabulate values for positions 0–3.

$$P = \begin{bmatrix} \sin(0) & \cos(0) & \sin(0.01) & \cos(0.01) \\ \sin(1) & \cos(1) & \sin(0.01) & \cos(0.01) \\ \sin(2) & \cos(2) & \sin(0.02) & \cos(0.02) \\ \sin(3) & \cos(3) & \sin(0.03) & \cos(0.03) \end{bmatrix} \approx \begin{bmatrix} 0.0000 & 1.0000 & 0.0000 & 1.0000 \\ 0.8415 & 0.5403 & 0.0100 & 0.9999 \\ 0.9093 & -0.4161 & 0.0200 & 0.9998 \\ 0.1411 & -0.9900 & 0.0300 & 0.9996 \end{bmatrix}$$

Each *row* of P is the positional encoding vector for one position; it is added element-wise to the corresponding token embedding X_{pos} .

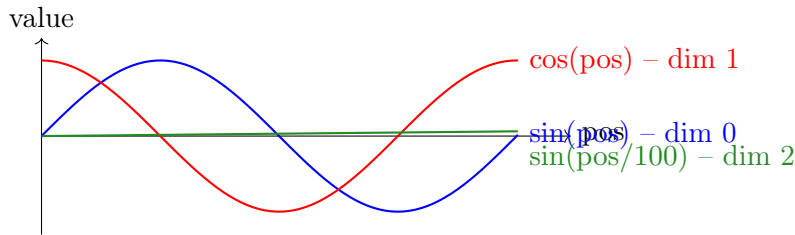


Figure 1: Sinusoidal positional encoding for $d_{\text{model}} = 4$. Dimensions 0–1 oscillate quickly with position (fine-grained, local information); dimension 2 (and 3, not shown, a phase-shifted cosine) oscillates so slowly over the range $[0, 6.3]$ that it appears almost flat — this is the intended behavior, since these dimensions are meant to distinguish positions over much longer ranges.

4.4 Two Key Properties

Why Sinusoids? Two Properties That Matter

The sinusoidal construction is not arbitrary; it provides two properties that a naive encoding (e.g., simply using the integer pos itself, or a random vector per position) would not:

1. **Bounded, smooth values.** Every entry of P_{pos} lies in $[-1, 1]$, regardless of pos . If we instead added the raw integer pos to the embedding, positions far into a long sequence (e.g., $\text{pos} = 5000$) would produce embeddings dominated by a huge additive constant, swamping the token’s semantic content and behaving very differently from the values seen during training on shorter sequences.
2. **Relative position is encoded as a linear transformation.** For a fixed offset k , the encoding $P_{\text{pos}+k}$ can be written as a *linear function* of P_{pos} (using the angle-addition identities for sine and cosine: $\sin(\text{pos} + k) =$

$\sin(\text{pos}) \cos(k) + \cos(\text{pos}) \sin(k)$, and analogously for cosine). For a fixed k , the terms $\cos(k)$ and $\sin(k)$ are constants, so $P_{\text{pos}+k}$ is a linear combination of P_{pos} 's components with position-independent coefficients. This means the *relative* displacement between two positions is encoded in a consistent, position-independent way — which plausibly helps the model learn “attend to the token two positions to the left” as a pattern that generalizes across the whole sequence, and even to sequence lengths longer than those seen during training.

4.5 Learned vs. Fixed Positional Encodings

Alternatives to Sinusoidal Encoding

The original Transformer used the fixed sinusoidal formula above, but several alternatives are common in modern models:

- **Learned positional embeddings.** Treat each position $0, \dots, n_{\text{max}} - 1$ as a “token” with its own trainable embedding vector, exactly like a word embedding (used in BERT and GPT-2). Simple, but cannot extrapolate beyond the maximum sequence length n_{max} seen during training.
- **Rotary Positional Embeddings (RoPE).** Instead of *adding* a positional vector, RoPE *rotates* the query and key vectors by an angle that depends on position, applied directly within the attention computation. This preserves the relative-position property in a more direct way and is widely used in modern large language models.
- **ALiBi (Attention with Linear Biases).** Adds a position-dependent penalty directly to the attention scores $QK^\top$ (penalizing attention to distant tokens), without modifying Q , K , or V at all.

All of these approaches address the same fundamental gap identified in Section 3.3: self-attention is permutation-equivariant and therefore cannot distinguish sequences that differ only by token order, except up to a corresponding permutation of the outputs. Something must supply position information explicitly.

Bridge from RNNs

It is illuminating to compare how RNNs and Transformers each encode position. In an RNN, position is encoded *implicitly and structurally*: token t 's influence on the output necessarily passes through exactly t recurrent steps, so “how far back” a piece of information corresponds directly to “how many matrix multiplications it has survived.” In a Transformer, position must be encoded *explicitly*, as an additional input signal, precisely *because* the architecture has removed the step-by-step structure that made position implicit. This is the price paid for parallelism: what used to be “free” (order, via the recurrence itself) must now be engineered in.

Entry Ticket 4

Work through the following before reading about residual connections and layer normalization.

1. For $d_{\text{model}} = 512$, compute the denominator $10000^{2i/d_{\text{model}}}$ for $i = 0$ and for $i = 255$ (the largest value of i). What is the ratio between these two denominators? What does this imply about the range of oscillation frequencies present across all 512 dimensions?
2. Using the angle-addition identity $\sin(\text{pos} + k) = \sin(\text{pos})\cos(k) + \cos(\text{pos})\sin(k)$, explain in your own words why $P_{\text{pos}+k}$ can be expressed as a linear function of P_{pos} (for fixed k). (You do not need to derive the full matrix; a one-paragraph qualitative explanation is sufficient.)
3. A learned positional embedding table has one row per position, up to $n_{\text{max}} = 512$. What happens, conceptually, if this model is given an input sequence of length 600 at inference time? Does the sinusoidal formula have the same problem?
4. The positional encoding is *added* to the token embedding ($Z_{\text{pos}} = X_{\text{pos}} + P_{\text{pos}}$), rather than concatenated. What potential downside could this additive combination have, if the magnitudes of X_{pos} and P_{pos} were very different? (Hint: think about what each term would have to compete with.)

5 Residual Connections and Layer Normalization

5.1 Motivation: Stacking Many Layers

A real Transformer is not a single attention computation; it is a *stack* of many identical layers (the original architecture used 6 encoder layers and 6 decoder layers; modern large language models use dozens to over a hundred). Just as the RNN notes observed that an unrolled RNN “behaves like a very deep network,” a stack of L Transformer layers *is* a very deep network — and deep networks are notoriously hard to train: gradients can vanish or explode as they propagate through many layers, and the optimization landscape becomes harder to navigate.

Two architectural components, used together in every Transformer layer, address this directly: **residual (skip) connections** and **layer normalization**.

5.2 Residual Connections

Residual Connection

Instead of a layer computing a direct transformation $Y = F(X)$, a residual connection reformulates the computation as

$$Y = X + F(X),$$

where F is the “main” transformation of the layer (e.g., self-attention, or a feed-forward network). The layer now only needs to learn the *residual* (the difference, or correction) $F(X)$ relative to the input X , rather than the full output from scratch.

Why this helps gradients flow. Differentiating $Y = X + F(X)$ with respect to X gives

$$\frac{\partial Y}{\partial X} = I + \frac{\partial F}{\partial X}.$$

The key insight is that the gradient path through Y now has *two components*: one that flows through F (which may be small or unstable early in training) and one that flows through the identity shortcut (always equal to I). This greatly reduces the risk of gradient vanishing through any single such layer. That said, a residual connection does not guarantee the absence of vanishing gradients in a deep stack: interactions across many layers, and the behavior of $\partial F/\partial X$ itself, still matter. Think of it as replacing a narrow bridge with a wide road: much easier to cross, but you still have to walk the full length of the network.

Bridge from RNNs

This should sound familiar. The LSTM notes presented the cell-state update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$ as a “gradient highway,” because $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$ is close to the identity when the forget gate is open. The residual connection $Y = X + F(X)$ is, in a precise sense, the *same idea applied across layers instead of across time steps*: an additive path that lets information (and gradients) bypass a transformation almost untouched. The key difference is that the LSTM’s highway is *gated* (the forget

gate can choose to close it), whereas the Transformer’s residual highway is *always fully open* ($\partial Y / \partial X$ always includes a full-strength I term, with no learned gate that could shut it). This makes residual connections simpler but also means the network must rely on $F(X)$ itself, and on normalization, to control how much new information is mixed in at each layer.

Why this helps optimization, intuitively. If, at some point during training, a particular layer’s transformation F is not yet useful (or is actively harmful), the network can effectively “skip” it by driving $F(X) \rightarrow 0$, in which case $Y \approx X$ — the layer behaves like an identity function and passes its input through unchanged. Without the residual connection, the layer would have to learn the identity function *exactly* via F , which is a comparatively difficult target for a nonlinear transformation to hit. With the residual connection, “do nothing” is the *default* (zero) output of F , making it an easy starting point from which the network can gradually learn useful corrections.

5.3 Layer Normalization

Residual connections solve the problem of gradient flow across layers, but they introduce a new issue: if every layer *adds* something to its input, the magnitude (scale) of the activations can grow without bound as the signal passes through many layers. **Layer normalization** addresses this by re-centering and re-scaling activations after each sub-layer.

Layer Normalization

Given an activation vector $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ (the representation of a *single token* at a *single layer*), layer normalization computes the mean and standard deviation *across the d features of that vector*:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 + \epsilon},$$

normalizes,

$$\hat{x}_i = \frac{x_i - \mu}{\sigma},$$

and applies a learned element-wise affine transformation,

$$y_i = \gamma_i \hat{x}_i + \beta_i,$$

where $\gamma = (\gamma_1, \dots, \gamma_d) \in \mathbb{R}^d$ and $\beta = (\beta_1, \dots, \beta_d) \in \mathbb{R}^d$ are learned parameter vectors (each element γ_i, β_i corresponds to one feature dimension), and ϵ is a small constant for numerical stability. These parameters are shared across all sequence positions but are specific to each layer.

Myth: Layer Normalization Is the Same as Batch Normalization

Both techniques normalize activations to have approximately zero mean and unit variance, but they normalize *across different axes*, and this difference matters enormously for sequence models. **Batch normalization** computes statistics across the *batch* dimension: for a given feature, it averages over all examples in the mini-batch. This makes batch normalization’s behavior dependent on batch composition and awkward for variable-length sequences (you would need to handle padding carefully, and statistics differ between training, where batches are large, and inference, where you might process one sequence at a time). **Layer normalization** computes statistics *independently for each token, across that token’s own features* — it never looks at other examples in the batch, nor at other positions in the sequence. This makes it naturally suited to the Transformer, where computation is per-token and highly parallel, and where sequence lengths vary from example to example.

5.4 Putting Them Together: The Transformer Sub-Layer Pattern

Every major component of a Transformer layer (self-attention, and a position-wise feed-forward network, introduced in Section 6) is wrapped in the same pattern:

Residual + LayerNorm Sub-Layer Pattern

$$Y = \text{LayerNorm}(X + \text{SubLayer}(X)), \quad (6)$$

where *SubLayer* is either a multi-head self-attention block or a feed-forward network. This is sometimes called the *post-norm* arrangement. (A common variant, *pre-norm*, applies LayerNorm to *X* *before* passing it to *SubLayer*, i.e., $Y = X + \text{SubLayer}(\text{LayerNorm}(X))$; many modern large language models use the pre-norm variant because it tends to be more stable for very deep stacks.)

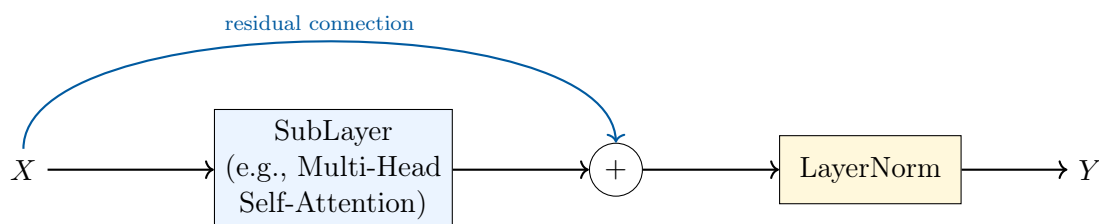


Figure 2: The residual + layer normalization sub-layer pattern. The input X follows two paths: one through the *SubLayer*, and one directly to the addition node (the residual shortcut, shown in blue). Their sum is then normalized by *LayerNorm* to produce Y . This pattern is applied around both the self-attention sub-layer and the FFN sub-layer in every Transformer block.

Architectural Stability as a Design Philosophy

Sections 3.1 and 5 together illustrate a unifying theme of the Transformer architecture: *stability is engineered into the forward pass*, rather than left to training-time interventions alone. The $\sqrt{d_k}$ scaling controls the spread of attention scores *before* the softmax; the residual connection provides a direct gradient path through each

sub-layer; and layer normalization controls the scale of activations *between* every sub-layer. These three mechanisms, applied uniformly throughout the network, are largely what makes it practical to stack dozens or hundreds of Transformer layers — something that would be extremely difficult for a vanilla deep MLP, an unrolled RNN, or even a stack of unmodified self-attention layers without these safeguards.

Entry Ticket 5

Reflect on residual connections and layer normalization before reading about the full encoder-decoder architecture.

1. Suppose a particular self-attention sub-layer, early in training, computes $F(X) \approx 0$ for all inputs (its weights have not yet learned anything useful). What is Y in Eq. 6 in this case? Why is this a *useful* starting point for training, compared to a layer without a residual connection (where $Y = F(X) \approx 0$ would simply destroy the signal)?
2. In the LSTM notes, the forget gate $\mathbf{f}_t$ provides a *learned, dynamic* control over how much of the past is preserved at each step. Does the residual connection $Y = X + F(X)$ provide an analogous learned control over how much of X is preserved versus how much of $F(X)$ is added? If not, what *does* provide some control over the relative contributions of X and $F(X)$?
3. Layer normalization computes μ and σ across the *feature* dimension of a single token's vector, not across the batch or across other tokens. If two different tokens in the same sequence have very different raw activation magnitudes before normalization, will they have similar magnitudes *after* layer normalization? What does this imply for how attention scores (computed *before* normalization, in the next layer's Q and K projections) might behave if normalization were removed?
4. A student claims: "Since $\partial Y / \partial X = I + \partial F / \partial X$ always includes the identity matrix, residual connections *completely* solve the vanishing gradient problem for Transformers, the same way the LSTM's cell state *completely* solves it for RNNs." Identify the overstatement in this claim. (Hint: the same kind of overstatement applies to LSTMs — look back at the LSTM misconception box in the RNN notes.)

6 Assembling the Architecture: Encoder, Decoder, and Cross-Attention

6.1 The Encoder Block

A single **Transformer encoder block** combines everything from Sections 2–5 into one reusable unit:

Transformer Encoder Block

$$X' = \text{LayerNorm}(X + \text{MultiHead}(X, X, X)), \quad (7)$$

$$X'' = \text{LayerNorm}(X' + \text{FFN}(X')), \quad (8)$$

where $\text{MultiHead}(X, X, X)$ denotes multi-head *self*-attention (queries, keys, and values all derived from X), and FFN is a position-wise feed-forward network, typically two linear layers with a ReLU (or GELU) activation in between:

$$\text{FFN}(x) = \max(0, x\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2, \quad \mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}.$$

A full Transformer encoder is a stack of N such blocks (e.g., $N = 6$ in the original architecture, $N = 12$ or 24 in BERT-base/large), each with its *own* parameters.

Why a Feed-Forward Network at All?

Self-attention mixes information *across positions* (each token’s representation is updated using information from other tokens), but the mixing is a *linear* combination of value vectors (Eq. 1), followed only by the linear output projection $\mathbf{W}^O$. The position-wise FFN, applied identically and independently to each position, is where the model gets its main *nonlinear, per-token* processing capacity — analogous to the role of nonlinear activations between layers in an MLP. A useful (if imperfect) mental model: self-attention decides *which* information to gather from where; the FFN decides *what to do* with the information once gathered, at each position independently.

6.2 The Decoder Block: Masked Self-Attention

Many tasks (machine translation, text generation, summarization) require *generating* an output sequence token by token. The Transformer **decoder** is designed for this autoregressive setting, and it differs from the encoder in one crucial way: **masked self-attention**.

Causal (Masked) Self-Attention

When generating token t , the model must not be allowed to “see” tokens $t+1, t+2, \dots$ — otherwise it could trivially “cheat” by attending directly to the future tokens it is supposed to predict. Masked self-attention enforces this by modifying the attention

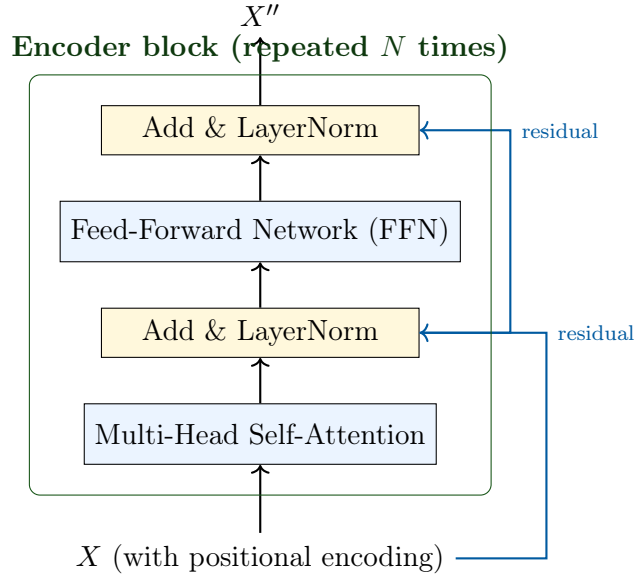


Figure 3: A single Transformer encoder block. Data flows bottom to top. For each sub-layer (self-attention and FFN), the residual connection (blue arrows) carries the sub-layer’s *input* forward, bypassing the sub-layer itself, to be added back at the “Add & LayerNorm” step. This implements $Y = \text{LayerNorm}(X + \text{SubLayer}(X))$ for both sub-layers.

score matrix $S = QK^\top / \sqrt{d_k}$ before the softmax:

$$S'_{ij} = \begin{cases} S_{ij} & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases}$$

After the row-wise softmax, $\exp(-\infty) = 0$, so $\alpha_{ij} = 0$ for all $j > i$: token i attends *only* to tokens at positions $\leq i$ (itself and everything before it).

Myth: Masking Means Future Tokens Are Removed from the Input

A natural but incorrect mental model is that masked self-attention somehow “deletes” or “hides” future tokens from the sequence. This is not how it works during training.

During training, the *entire* target sequence (e.g., the full translated sentence) is fed into the decoder *at once*, as a single matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$ — this is what enables the parallelization that motivated the whole architecture (Section 1.1)! The masking operates *only on the attention score matrix* S , zeroing out (via $-\infty$ before softmax) the specific entries α_{ij} for $j > i$. Every token’s representation is computed in parallel, but each token’s attention pattern is restricted to only “look backward.” Table 1 summarizes how this changes at inference time.

6.3 Two Types of Masks: A Quick Clarification

Before introducing cross-attention, it is worth pausing to distinguish the two masking mechanisms that appear in Transformer implementations. These are easy to confuse,

Table 1: Training vs. inference behavior of the decoder.

Situation	Input to decoder	Causal mask?	Parallel positions?
Training	Full target sequence (shifted right), fed all at once	Yes — applied to attention scores	Yes
Inference	Tokens generated so far, one at a time	Yes / implicit (no future tokens exist)	No, one step at a time

because PyTorch uses overlapping argument names for them.

Causal Mask vs. Padding Mask

- **Causal (attention) mask** answers: “*which positions am I not allowed to attend to?*” It has shape (n, n) and is applied to the attention score matrix S . Used in all decoders to enforce the left-to-right constraint. In PyTorch: the `attn_mask` argument to `nn.MultiheadAttention`, or the `mask` argument to `nn.TransformerEncoder`.
- **Padding mask** answers: “*which positions in the sequence are padding tokens (not real content) and should be ignored?*” It has shape (B, n) where B is the batch size. Sequences in a batch are often of different lengths and padded to the same length; these padding positions should not contribute to attention. In PyTorch: `src_key_padding_mask` or `tgt_key_padding_mask`.

Both masks solve different problems and often coexist in a single model. A common debugging error is to swap them: the padding mask will then block real tokens, and the causal mask will allow attention to padding. Always verify that the model cannot attend to padding positions by checking that the corresponding attention weights are near zero.

6.4 Cross-Attention: Connecting Encoder and Decoder

For sequence-to-sequence tasks (e.g., translating an English sentence into Portuguese), the decoder needs access to the *input* sequence (processed by the encoder) while generating the *output* sequence. This is achieved with a second attention sub-layer in each decoder block: **cross-attention**.

Cross-Attention

Cross-attention uses the same formula as self-attention (Eq. 1), but with queries coming from the *decoder’s* current representations X_{dec} , while keys and values come from the *encoder’s* final output X_{enc} :

$$\text{CrossAttention}(X_{\text{dec}}, X_{\text{enc}}) = \text{Attention}(X_{\text{dec}} \mathbf{W}_Q, X_{\text{enc}} \mathbf{W}_K, X_{\text{enc}} \mathbf{W}_V).$$

Conceptually: “*for each token I am currently generating (query), which parts of the source sentence (keys/values) are most relevant?*” This is a direct, learned, content-

based generalization of the word-alignment idea from classical statistical machine translation.

Bridge from RNNs

This resolves the misconception flagged early in Section 2: in self-attention, Q , K , and V all come from the same sequence. In *cross*-attention, Q comes from one sequence (the decoder’s output-so-far) and K, V come from a *different* sequence (the encoder’s representation of the source). Note also the connection to the encoder-decoder RNN configuration from the RNN notes: there, the entire source sequence was compressed into a *single* fixed-size context vector $\mathbf{c}$, which initialized the decoder — another instance of the lossy-summary bottleneck. Cross-attention removes this bottleneck: the decoder can attend to *any* position in the encoder’s output, with weights that can differ for every token it generates, rather than relying on one shared summary vector for the entire output sequence.

6.5 The Full Encoder-Decoder Transformer

Decoder Block (Full)

Each decoder block consists of *three* sub-layers, each wrapped in the residual + LayerNorm pattern of Eq. 6:

1. **Masked multi-head self-attention** over the decoder’s own (target) sequence so far.
2. **Multi-head cross-attention**, with queries from the decoder and keys/values from the encoder’s output.
3. **Position-wise feed-forward network**, identical in structure to the encoder’s FFN.

Entry Ticket 6

Consolidate the full architecture before reading about model families and implementation.

1. In the encoder block (Eq. 7–8), is self-attention masked or unmasked? Why is the choice different from the decoder’s first sub-layer?
2. A decoder block has *three* sub-layers, while an encoder block has *two*. Name the extra sub-layer and explain, in one sentence, what information it allows the decoder to access that it otherwise would not have.
3. During *training* of a machine translation model, the full target sentence (in Portuguese, say) is available. Explain why masked self-attention is still necessary in this case — i.e., why can’t the decoder simply use *unmasked* self-attention over the full target sentence during training, even though the full sentence is available?
4. Consider the sentence pair from the RNN notes’ encoder-decoder discussion: an English sentence (source) and its Portuguese translation (target), of possibly different lengths $n_{\text{src}} \neq n_{\text{tgt}}$. In cross-attention, what are the shapes of Q , K , and V ? (Express them in terms of n_{src} , n_{tgt} , and d_k .) What does this

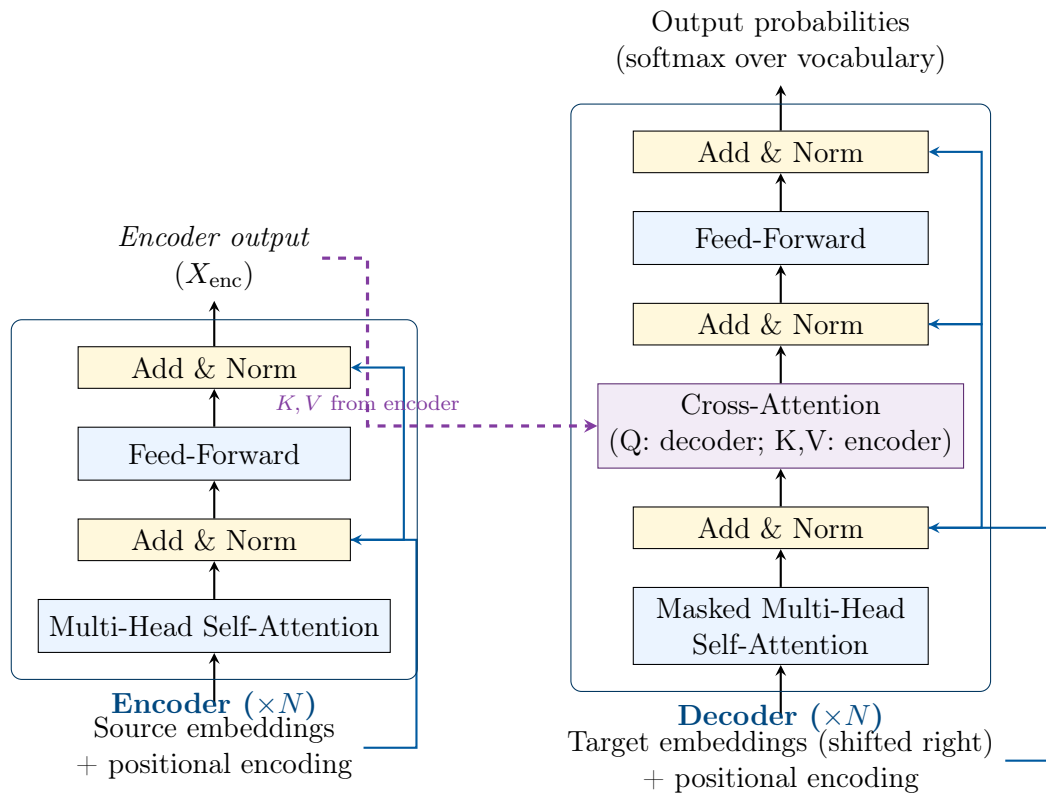


Figure 4: The encoder-decoder Transformer architecture. Data flows bottom to top in each column. The encoder (left) processes the source sequence: self-attention gathers context, then the FFN applies per-token nonlinear processing, both wrapped in residual + Norm. The encoder's final output X_{enc} (the contextualized representation of every source token) is passed as keys K and values V into the cross-attention sub-layer of *every* decoder block. The decoder (right) uses masked self-attention over the generated tokens so far, then cross-attention to consult the encoder output, then a FFN. Blue arrows denote residual connections; residuals are omitted from the encoder side for clarity but follow the same pattern.

imply about the shape of the cross-attention weight matrix α ?

7 Transformer Families: Encoder-Only, Decoder-Only, Encoder-Decoder

7.1 Not All Transformers Are Encoder-Decoder

The architecture described in Section 6 — a full encoder stack connected to a full decoder stack via cross-attention — is one design point in a family of Transformer variants. Modern practice has produced two additional architectures, each tailored to a different class of tasks.

Three Transformer Families

- **Encoder-only** (e.g., BERT [Devlin et al., 2019]): uses the encoder stack only, with *bidirectional* (unmasked) self-attention. Every token can attend to every other token in both directions. Designed for tasks that require understanding a *complete* input (classification, named entity recognition, extractive QA). Not designed for autoregressive generation: without a causal constraint, these models do not naturally generate sequences token by token.
- **Decoder-only** (e.g., GPT family [Radford et al., 2018]): uses the decoder stack only, with *causal* (masked) self-attention and *no* encoder or cross-attention. At each position, a token can attend only to itself and to tokens that came before it. Designed for autoregressive *generation*: predict the next token given all previous tokens.
- **Encoder-decoder** (e.g., T5 [Raffel et al., 2020], original Transformer [Vaswani et al., 2017]): the full architecture from Section 6. Takes an input sequence (encoder), generates an output sequence (decoder), with cross-attention connecting them. Designed for *conditional generation*: tasks where the output is a sequence that depends on an input sequence, such as translation, summarization, and abstractive QA.

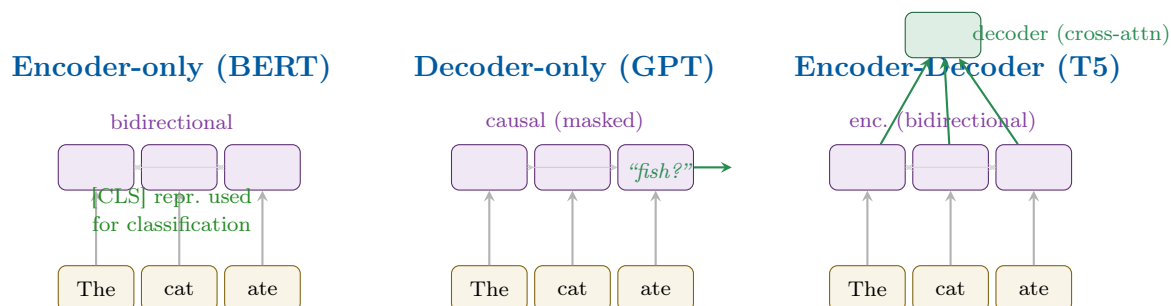


Figure 5: The three Transformer families. **Encoder-only:** all tokens attend to all other tokens (bidirectional); outputs used for classification/understanding tasks. **Decoder-only:** each token attends only to itself and prior tokens (causal masking); the final representation is projected to vocabulary logits to predict the next token. **Encoder-decoder:** full encoder with bidirectional attention feeds into a decoder via cross-attention; used for conditional generation. Arrows represent attention connections; not all are shown.

7.2 Architectural Differences in Detail

Table 2 summarizes the key differences.

Table 2: Comparison of the three Transformer families.

Property	Encoder-only	Decoder-only	Encoder-decoder
Self-attention type	Bidirectional (un-masked)	Causal (masked)	Enc: bidir.; Dec: masked
Cross-attention	None	None	Decoder attends to encoder
Representative models	BERT, RoBERTa, Distil-BERT	GPT-2/3/4, LLaMA, Mistral	T5, BART, original Transformer
Primary use cases	Classification, NER, extractive QA	Language generation, few-shot prompting	Translation, summarization, seq2seq
Naturally generates text?	No	Yes	Yes
Trained with	Masked language modeling	Next-token prediction	Seq2seq (e.g., cross-entropy on target)

Why Decoder-Only Models Have Become Dominant

The modern trend in large language models (GPT-3, GPT-4, LLaMA, Mistral, Claude, Gemini) is to use a *decoder-only* architecture at massive scale. One key reason is versatility: almost any task can be reformulated as text generation (classification becomes “generate a class label”; translation becomes “generate the target sentence”; QA becomes “generate the answer”). A single decoder-only model can handle all of these via appropriate prompting, without architectural changes. Encoder-decoder models require the input/output decomposition to be specified during architecture design, which is less flexible.

A second reason is training efficiency: decoder-only models are trained with a simple *next-token prediction* (causal language modeling) objective on vast unlabeled text corpora, with no need to label data or define separate encoder/decoder inputs. This simplicity at scale has proved extremely powerful.

Myth: “Attention Is All You Need” Means the Transformer Has No Linear Layers

The title of Vaswani et al. [2017] refers to the fact that the Transformer replaces *recurrence* (RNNs, LSTMs) and *convolution* (CNNs for sequences) with attention. It does not mean the model is attention-only. A Transformer contains a large number of standard linear (dense) layers: the Q , K , V projection matrices ($\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$), the output projection $\mathbf{W}^O$, the two-layer FFN ($\mathbf{W}_1$, $\mathbf{W}_2$), the embedding matrix, and the final unembedding (vocabulary projection) layer. In fact, in modern large language models, the FFN layers account for roughly *two-thirds* of the total parameter count, with the attention projection matrices making up most of the remainder.

Bridge from RNNs

Looking back at the full progression: MLP $\rightarrow$ CNN $\rightarrow$ RNN $\rightarrow$ Transformer, each step has been about identifying a structural property of the data and encoding it as an *architectural inductive bias*:

- The MLP imposes no structure: any input $\rightarrow$ output mapping is in principle learnable.
- The CNN imposes *local connectivity* and *spatial parameter sharing*: nearby inputs are more likely related; the same pattern means the same thing wherever it appears in an image.
- The RNN imposes *temporal parameter sharing* and *sequential processing*: the same computation applies at each time step; the history is compressed into a running state.
- The Transformer imposes *pairwise interaction* and *global connectivity*: every element can directly influence every other; order is injected externally rather than structurally. The inductive bias is minimized (the model is given maximal freedom in how it routes information), which is both the Transformer's great strength at scale and a reason it can require substantially more data to learn from than the more constrained architectures.

Entry Ticket 7

Consolidate the Transformer family comparison.

1. You are building a system that reads customer support tickets (variable-length, typically 50–300 words) and classifies each ticket into one of 10 categories. Which Transformer family would you use? Would you use masked or unmasked self-attention? Justify your choice.
2. A colleague proposes using a decoder-only model for the same task, reformulating classification as generation (“What category is this ticket? Category:”, and then generating the label). Name one potential advantage and one potential disadvantage of this approach compared to using an encoder-only model.
3. In the encoder-decoder architecture, the encoder is run *once* per input sequence, and its output is reused by *all* decoder blocks at *every* generation step. Why is this computationally efficient? What would be wasteful about running the encoder anew for each generated token?
4. Consider a decoder-only model generating a 100-token response. At the 100th generation step, token 100 attends to all 99 previous tokens via masked self-attention. In an RNN, what would play the analogous role at step 100, and how is the information from all 99 previous tokens represented at that step? What fundamental structural difference makes Transformers generally better at exploiting this long-range context?

8 Implementation in PyTorch

8.1 Two Types of Masks: PyTorch Conventions

Before showing code, a brief note on PyTorch’s masking API, since this is one of the most common sources of bugs. (The conceptual distinction was introduced in Section 6; here we map it to PyTorch argument names.)

In `nn.MultiheadAttention`:

- **attn_mask** (shape: (n, n) or $(B \cdot h, n, n)$): the **causal or custom attention mask**. A value of `True` (boolean) or $-\infty$ (float) at position (i, j) means “token i is not allowed to attend to token j .”
- **key_padding_mask** (shape: (B, n)): the **padding mask**. A value of `True` at (b, j) means “position j in sequence b is a padding token and should be ignored.”

When using `nn.TransformerEncoder`, the equivalent arguments are `mask` (causal) and `src_key_padding_mask` (padding). Always verify mask behavior by inspecting the resulting attention weight matrices on a known example.

8.2 PyTorch Transformer Modules

PyTorch provides ready-to-use modules for all the components discussed in these notes. Table 3 lists the most important ones.

Table 3: Key PyTorch modules for Transformer implementation.

Module	What it implements
<code>nn.MultiheadAttention</code>	Multi-head attention (Eq. 2–3), including $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$
<code>nn.LayerNorm</code>	Layer normalization (Section 5)
<code>nn.Transformer-EncoderLayer</code>	One encoder block (MHA + FFN, both with residuals and LayerNorm)
<code>nn.TransformerEncoder</code>	Stack of N encoder blocks
<code>nn.Transformer-DecoderLayer</code>	One decoder block (masked MHA + cross-attn + FFN)
<code>nn.TransformerDecoder</code>	Stack of N decoder blocks
<code>nn.Transformer</code>	Full encoder-decoder Transformer
<code>nn.Embedding</code>	Token embedding lookup (same as in RNN notes)

8.3 A Minimal Self-Attention Layer from Scratch

Before using PyTorch’s higher-level modules, it is instructive to implement the core scaled dot-product attention (Eq. 1) directly. Listing 1 does this as a clean, standalone function.


```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 def scaled_dot_product_attention(Q, K, V, mask=None):
7     """
8     Args:
9         Q: (batch, n_queries, d_k)
10        K: (batch, n_keys, d_k)
11        V: (batch, n_keys, d_v)
12        mask: (batch, n_queries, n_keys) boolean mask;
13              True = keep, False = mask out (set to -inf)
14
15    Returns:
16        output: (batch, n_queries, d_v)
17        attn_weights: (batch, n_queries, n_keys)
18    """
19    d_k = Q.size(-1)
20    # Step 1: compatibility scores
21    scores = torch.bmm(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
22    # Step 2: apply causal or padding mask if provided
23    if mask is not None:
24        scores = scores.masked_fill(~mask, float('-inf'))
25    # Step 3: row-wise softmax -> attention weights
26    attn_weights = F.softmax(scores, dim=-1)
27    # Step 4: weighted sum of values
28    output = torch.bmm(attn_weights, V)
29    return output, attn_weights
30
31 # Quick shape check
32 B, n, d_k, d_v = 2, 5, 16, 16
33 Q = torch.randn(B, n, d_k)
34 K = torch.randn(B, n, d_k)
35 V = torch.randn(B, n, d_v)
36 out, attn = scaled_dot_product_attention(Q, K, V)
37 print(f"Output: {out.shape}")          # (2, 5, 16)
38 print(f"Weights: {attn.shape}")        # (2, 5, 5)
39 print(f"Row sums (should be 1): {attn[0].sum(dim=-1)}")

```

Listing 1: Scaled dot-product attention from scratch in PyTorch.

8.4 Transformer Encoder for Text Classification

Listing 2 implements a complete, trainable Transformer encoder for sequence classification (many-to-one), using PyTorch's `nn.TransformerEncoder`. Compare the structure to the `LSTMClassifier` from the RNN notes: the external API is nearly identical, but the internal computation is fundamentally different.

```

1 class TransformerClassifier(nn.Module):
2     def __init__(self, vocab_size, d_model, n_heads, d_ff,

```

```

3         n_layers, n_classes, max_len=512, dropout=0.1):
4     super().__init__()
5     self.embedding = nn.Embedding(vocab_size, d_model,
6                                   padding_idx=0)
7     # Learned positional embedding (alternative to sinusoidal)
8     self.pos_embedding = nn.Embedding(max_len, d_model)
9     self.dropout = nn.Dropout(dropout)
10
11    # Stack of N encoder blocks
12    enc_layer = nn.TransformerEncoderLayer(
13        d_model=d_model,
14        nhead=n_heads,
15        dim_feedforward=d_ff,
16        dropout=dropout,
17        batch_first=True      # input: (batch, seq, d_model)
18    )
19    self.encoder = nn.TransformerEncoder(enc_layer,
20                                         num_layers=n_layers)
21
22    self.classifier = nn.Linear(d_model, n_classes)
23
24    def forward(self, x, padding_mask=None):
25        # x: (batch, seq_len) integer token IDs
26        B, T = x.shape
27        positions = torch.arange(T, device=x.device).unsqueeze(0)
28        # (1, T)
29
30        # Embed tokens and add positional encoding
31        emb = self.dropout(self.embedding(x) + self.pos_embedding(
32            positions))
33
34        # Encode: (batch, seq_len, d_model)
35        # padding_mask: True where a token is padding (should be
36        # ignored)
37        enc_out = self.encoder(emb, src_key_padding_mask=
38            padding_mask)
39
40        # Many-to-one: use representation of the first token (like
41        # [CLS] in BERT)
42        cls_repr = enc_out[:, 0, :]      # (batch, d_model)
43        return self.classifier(self.dropout(cls_repr)) # (batch,
44            n_classes)
45
46    # Usage
47    model = TransformerClassifier(
48        vocab_size=10000, d_model=128, n_heads=4,
49        d_ff=512, n_layers=3, n_classes=2
50    )
51    criterion = nn.CrossEntropyLoss()
52    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
53                                    weight_decay=1e-2)

```

```

46 print(f"Parameters: {sum(p.numel() for p in model.parameters()):,}")
    )

```

Listing 2: Transformer encoder for text classification in PyTorch.

The [CLS]

Token Convention] In Listing 2, the representation of the *first* token (position 0) is used as a global summary of the sequence for classification. This follows the BERT convention, where a special [CLS] (“classify”) token is prepended to every input sequence. Because the encoder is *bidirectional* (unmasked), the [CLS] token at position 0 can attend to all other tokens from the very first layer; by the output of the last encoder layer, its representation is a rich aggregate of the entire sequence. In practice, for the [CLS] representation to be useful, either (a) the model must be pre-trained with an objective that forces the [CLS] token to capture global information (as in BERT’s masked language modeling + next-sentence prediction), or (b) the entire model must be fine-tuned end-to-end on the downstream task so that the [CLS] pathway is trained to be informative for classification.

8.5 Building a Causal Language Model (Decoder-Only)

Listing 3 implements a minimal decoder-only language model, analogous to the character-level language model from the RNN notes (Listing 2 there). The key difference is the use of a causal attention mask.

```

1 class CausalLM(nn.Module):
2     def __init__(self, vocab_size, d_model, n_heads, d_ff,
3                   n_layers, max_len=1024, dropout=0.1):
4         super().__init__()
5         self.embedding = nn.Embedding(vocab_size, d_model,
6                                       padding_idx=0)
7         self.pos_embedding = nn.Embedding(max_len, d_model)
8         self.dropout = nn.Dropout(dropout)
9
10        # TransformerEncoder is reused here -- causal masking is
11        # applied below
12        enc_layer = nn.TransformerEncoderLayer(
13            d_model=d_model, nhead=n_heads, dim_feedforward=d_ff,
14            dropout=dropout, batch_first=True
15        )
16        self.transformer = nn.TransformerEncoder(enc_layer,
17                                                  num_layers=
18                                                  n_layers)
19
20        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
21        # Tie embedding and unembedding weights (common practice)
22        self.lm_head.weight = self.embedding.weight
23
24    def forward(self, x):
25        B, T = x.shape
26        positions = torch.arange(T, device=x.device).unsqueeze(0)

```

```

23     emb = self.dropout(self.embedding(x) + self.pos_embedding(
24         positions))
25
26     # Build causal mask: True means "this position is masked (
27     # cannot attend)"
28     # nn.TransformerEncoder uses True = mask out
29     causal_mask = torch.triu(
30         torch.ones(T, T, device=x.device, dtype=torch.bool),
31         diagonal=1
32     )
33     out = self.transformer(emb, mask=causal_mask,
34                             is_causal=True) # hint for flash
35                                             attention
36     logits = self.lm_head(out) # (batch, seq_len, vocab_size)
37     return logits
38
39 def train_step(model, batch, optimizer, criterion):
40     # batch: (B, T+1); input = batch[:, :-1], target = batch[:, 1:]
41     inputs, targets = batch[:, :-1], batch[:, 1:]
42     optimizer.zero_grad()
43     logits = model(inputs) # (B, T, V)
44     loss = criterion(logits.reshape(-1, logits.size(-1)),
45                     targets.reshape(-1))
46
47     loss.backward()
48     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
49     optimizer.step()
50     return loss.item()

```

Listing 3: Minimal decoder-only causal language model in PyTorch.

AdamW vs. Adam for Transformers

The RNN notes used `torch.optim.Adam`. The standard optimizer for Transformers is `torch.optim.AdamW`, which adds *weight decay* (L2 regularization) correctly decoupled from the adaptive learning-rate scaling. The original Adam applies weight decay incorrectly (as gradient regularization rather than true L2 decay), which matters at scale. For Transformers, a **learning rate warmup schedule** is also strongly recommended: start with a very small learning rate, ramp up linearly for the first few thousand steps, then decay (typically following a cosine or inverse-square-root schedule). Without warmup, the Adam adaptive estimates are unreliable in early training and can send parameters in harmful directions from which recovery is slow.

9 Computational Complexity and Efficient Attention

9.1 The $O(n^2)$ Bottleneck

The key computational cost of self-attention is the construction and multiplication of the attention weight matrix $\alpha \in \mathbb{R}^{n \times n}$. Both storing this matrix and computing $QK^\top$ require:

$$\text{Time and Memory: } O(n^2 \cdot d_k) \approx O(n^2),$$

where n is the sequence length. This quadratic dependence on n is the defining bottleneck of the standard Transformer.

Table 4: Comparison of per-layer computational and memory costs for RNNs and Transformers.

Property	RNN (LSTM)	Transformer
Time per layer (training)	$O(n \cdot d^2)$ sequential	$O(n^2 \cdot d)$ parallelizable
Memory per layer	$O(n \cdot d)$	$O(n^2 + n \cdot d)$
Parallel across time?	No	Yes
Max path length (input $\rightarrow$ output)	$O(n)$ steps	$O(1)$ layers

The Fundamental Trade-off Between RNNs and Transformers

The RNN is *sequentially efficient* (each step is $O(d^2)$, and steps must be processed in order) but creates an $O(n)$ -length gradient path from the last output to the first input, making long-range learning difficult. The Transformer is *parallelism-efficient* (all positions are processed simultaneously) with an $O(1)$ -length gradient path between any two positions (via direct attention), but pays an $O(n^2)$ memory and compute cost for the attention matrix. For moderate n (up to a few thousand), this trade-off strongly favors the Transformer on modern GPUs, because the $O(n^2)$ cost is easily parallelized and GPUs excel at matrix multiplications. For very large n (tens of thousands of tokens or more), the $O(n^2)$ cost becomes prohibitive.

9.2 Efficient Attention Variants

The quadratic cost of full self-attention has motivated a large body of research on approximations and alternatives. The most important in current practice are:

- **FlashAttention** [Dao et al., 2022]. Does *not* reduce the asymptotic complexity; it remains $O(n^2)$. Instead, it reorders the computation of Eq. 1 to minimize expensive reads and writes between GPU high-bandwidth memory (HBM) and the faster on-chip SRAM, dramatically reducing the *wall-clock* time and memory footprint without any approximation. To understand why this is possible: even though the total number of arithmetic operations is the same, moving data between HBM and

SRAM is orders of magnitude slower than performing arithmetic on data already in SRAM. By tiling the computation so that large chunks fit in SRAM and are fully processed before being written back, FlashAttention avoids redundant memory traffic. The result is a substantial practical speedup even at the same theoretical $O(n^2)$ complexity. FlashAttention is now widely used in modern production Transformer systems.

- **Sliding-window (local) attention.** Each token attends only to a window of w neighboring tokens, reducing complexity to $O(n \cdot w)$. Long-range attention is provided either by a sparse set of global tokens or by hierarchical structure. Used in Longformer, BigBird.
- **Linear attention** (e.g., Performer). Approximates the softmax attention via a feature-map decomposition $\text{softmax}(QK^\top)V \approx \phi(Q)(\phi(K)^\top V)$, where ϕ is a low-dimensional feature map. By computing $\phi(K)^\top V$ first (a $d_k \times d_v$ matrix), the complexity drops to $O(n \cdot d_k \cdot d_v)$, linear in n . Quality degrades somewhat relative to full attention.
- **KV caching** (for inference only). During autoregressive generation, at each new step the model needs to compute only the Q , K , V vectors for the most recently generated token; the K and V vectors from all previous tokens are stored (“cached”) and reused. This avoids recomputing key/value projections for the entire growing context from scratch at every step. The per-step cost drops from $O(n^2)$ (if recomputing everything) to $O(n \cdot d)$, at the cost of $O(n \cdot d)$ memory for the cache. Note that autoregressive decoding remains inherently sequential (each token depends on all previous ones), so generating a 100-token response still requires 100 sequential steps; KV caching makes each step faster by eliminating redundant recomputation, not by reducing the number of steps.

KV Cache in Practice

KV caching is the reason that each individual decoding step costs roughly the same regardless of how many tokens have already been generated (up to memory limits), whereas recomputing the full context from scratch at each step would make later steps much more expensive. Modern inference frameworks (vLLM, TensorRT-LLM, and related systems) implement KV caching and associated optimizations (paged attention, prefix sharing) as their primary efficiency mechanism. Understanding KV caching is therefore essential context for deploying Transformer-based models, even if the training-time concerns around FlashAttention and efficient attention variants are more relevant for model development.

Myth: The Transformer’s $O(n^2)$ Cost Limits Context Length to a Few Hundred Tokens

This was true of early Transformers (GPT-2’s context window was 1024 tokens; the original BERT used 512). But with FlashAttention and associated engineering improvements (mixed-precision training, gradient checkpointing, sequence parallelism), modern models routinely process context windows of 8k, 32k, and 128k tokens or more in practice. The $O(n^2)$ scaling is a challenge, not a fundamental barrier. For most NLP tasks with inputs under 8k tokens, the quadratic cost is not

the practical bottleneck; model depth and parameter count usually are.

Entry Ticket 8

Reflect on complexity and practical considerations before the final section.

1. For an RNN with hidden size $d = 512$ and a Transformer with $d_{\text{model}} = 512$, at what sequence length n does the Transformer's attention matrix computation ($O(n^2 \cdot d)$ operations) become approximately as expensive as the RNN's full sequential processing ($O(n \cdot d^2)$ operations)? Solve for n symbolically, then substitute $d = 512$.
2. KV caching stores all previous K and V vectors in memory during autoregressive generation. If $d_k = d_v = 64$, and there are 12 layers with 8 heads each, how many floating-point numbers must be stored in the KV cache for a context of 4096 tokens? (Express your answer as a formula and evaluate it.)
3. A student proposes evaluating a Transformer encoder on a document of 50,000 words by splitting it into 50 chunks of 1,000 tokens each and encoding them independently. What important information is lost by this approach? What property of standard self-attention makes it impossible to handle the full 50,000-token document in one pass on typical hardware?
4. FlashAttention does not reduce the asymptotic $O(n^2)$ complexity but dramatically reduces wall-clock time. Explain conceptually how this is possible, using the GPU memory hierarchy (HBM vs. on-chip SRAM) as your framework.

10 Common Pitfalls, Debugging, and Final Remarks

10.1 Pitfalls Specific to Transformers

Many of the debugging practices from the RNN notes (shape checks, overfit-one-batch test, gradient norm monitoring) apply directly. Table 5 lists additional pitfalls specific to Transformer implementations.

10.2 The “Three Lines That Destroy a Transformer”

In practice, the most common subtle bugs in Transformer implementations come down to three issues:

1. **Forgetting positional encoding.** Without it, the model is permutation-equivariant and cannot learn any order-dependent patterns. The symptom is often a model that trains (loss decreases) but performs poorly because it learned order-invariant statistics only.
2. **Wrong masking convention.** PyTorch’s `nn.MultiheadAttention` uses `True = ignore` for `key_padding_mask` but `True = mask out` for `attn_mask` in float mode. Getting these backwards silently masks the wrong positions. Always verify that the model cannot see padding tokens by checking that attention weights on padding positions are near zero.
3. **Using Adam instead of AdamW with a warmup schedule.** Adam (without weight decay correction) can produce unstable early training trajectories in Transformers. A model that fails to learn in the first 1000 steps rarely recovers. Use AdamW with a warmup.

10.3 Summary of Key Concepts

The following points summarize the essential content of these notes.

- RNNs have two fundamental limitations: *sequential computation* (cannot parallelize across time) and a *fixed-size information bottleneck* (the hidden state). The Transformer addresses both by replacing recurrence with *self-attention*: direct, content-based, pairwise interactions between all tokens.
- **Scaled dot-product attention** computes, for each token, a weighted combination of all value vectors V_j , with weights given by the softmax of the scaled dot product between query Q_i and all keys K_j : $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$.
- The $\sqrt{d_k}$ **scaling** prevents softmax saturation (caused by large spread among dot products as d_k grows) and the resulting vanishing gradients through attention.
- **Multi-head attention** runs h parallel attention computations in smaller subspaces $d_k = d_{\text{model}}/h$, allowing different heads to specialize in different types of relationships.
- Self-attention is *permutation-equivariant*: it has no inherent notion of token order (permuting the input permutes the output correspondingly, but does not destroy the relationship between input and output positions). **Positional encoding** injects

order back in, by adding position-dependent vectors (sinusoidal or learned) to token embeddings before any attention layer.

- **Residual connections** ($Y = X + F(X)$) provide a direct gradient path through each sub-layer, greatly reducing the risk of vanishing gradients, analogous to the LSTM's cell-state highway through time. **Layer normalization** controls activation scale between sub-layers.
- A **Transformer encoder block** applies multi-head self-attention and a position-wise FFN, each wrapped in residual + LayerNorm. A full encoder is a stack of N such blocks.
- A **decoder block** adds *masked* (causal) self-attention (to prevent attending to future tokens) and *cross-attention* (to attend to the encoder's output).
- The three Transformer families — encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5) — differ in their attention masking and cross-attention structure, and are suited to understanding, generation, and conditional generation tasks, respectively.
- Standard self-attention has $O(n^2)$ time and memory cost in the sequence length n . **FlashAttention** reduces wall-clock cost via GPU memory-hierarchy optimization; **KV caching** enables efficient autoregressive inference by avoiding recomputation of previous keys and values.

10.4 Recommended Next Steps

These notes prepare the student for the following topics in the deep learning module:

MLP → CNNs → RNNs → **Transformers** → **Pre-training & Fine-tuning (BERT, GPT)** → Generative Models & LLMs

Key Takeaways

- Self-attention solves the RNN's bottlenecks by letting every token attend directly to every other token, enabling parallelism and eliminating the recurrent hidden-state bottleneck. The price paid is $O(n^2)$ attention cost.
- The $\sqrt{d_k}$ scaling, multi-head structure, positional encoding, residual connections, and layer normalization are not independent choices: each addresses a specific failure mode that would arise without it, and together they make deep Transformers trainable.
- Positional encoding is architecturally necessary; without it, self-attention is permutation-equivariant and cannot model word order: sequences that differ only by token order would produce the same (permuted) outputs.
- Masked self-attention enables parallel training of autoregressive models (all target tokens trained simultaneously) while enforcing the causal constraint (no peeking at future tokens).
- The decoder-only architecture (GPT lineage) has become dominant in large language models because next-token prediction on unlabeled text scales gracefully, and because almost any task can be framed as generation.

You are designing a Transformer-based system to assist doctors by reading clinical notes (typically 200–2000 words) and generating a structured discharge summary (typically 50–300 words). The discharge summary must accurately reflect the clinical note and must not invent clinical facts.

1. Which Transformer family (encoder-only, decoder-only, or encoder-decoder) would you choose for this task, and why? Justify your answer by explaining what each family cannot do in this setting.
2. For the attention in your chosen architecture: which sub-layers use bidirectional attention, and which use causal masking? Explain why each choice is correct for its respective purpose.
3. Clinical notes vary enormously in length. A note for a complex patient might be 4000 tokens, requiring an attention matrix of $4000 \times 4000 = 16$ million entries. Describe one practical technique (from Section 9) that would allow your model to handle such inputs without running out of GPU memory.
4. At inference time, the system generates one token of the summary at a time. Explain, concretely, how KV caching reduces the per-step compute cost compared to recomputing all attention from scratch, and identify which parts of the architecture benefit from KV caching (encoder, decoder self-attention, decoder cross-attention, or some subset).
5. A medical licensing board is concerned that the model might “hallucinate” clinical facts not present in the note. From an architectural perspective, explain the role of cross-attention in grounding the decoder’s output in the encoder’s representation of the input. Why is this grounding imperfect, and what additional training or inference strategies (beyond architectural choice) might reduce hallucination?

10 References

- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423>.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf. Technical Report.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. In *Journal of Machine Learning Research*, volume 21, pages 1–67, 2020. URL <https://jmlr.org/papers/v21/20-074.html>.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>.

Table 5: Common symptoms, likely causes, and fixes when implementing Transformers in PyTorch.

Symptom	Likely cause	Fix
Cross-entropy stays near $\log V$ (random baseline perplexity near V) forever	Missing or wrong causal mask; positional encoding absent	Verify <code>attn_mask</code> shape and value; add positional encoding
Identical outputs for different input sequences	Model is ignoring token identities (e.g., embedding not connected)	Check that token IDs are integers, not floats; verify embedding layer
Loss diverges early in training	Learning rate too high; no warmup	Use AdamW with warmup (e.g., linear warmup for 4k steps, then cosine decay)
Good train loss, poor val loss (overfitting)	Not enough data; model too large	Increase dropout; reduce d_{model} or n_{layers} ; add more data
Two input sequences of same token set but different order produce same output	Positional encoding missing or not added before encoder	Add positional encoding: <code>emb = token_emb + pos_enc</code>
<code>RuntimeError: mask shape mismatch</code>	Padding mask vs. causal mask confusion	Padding mask shape (B, T); causal mask shape (T, T); check <code>src_key_padding_mask</code> vs. <code>attn_mask</code>
Very slow training despite GPU available	Attention not using memory-efficient kernel	Use PyTorch ≥ 2.0 with <code>scaled_dot_product_attention</code>
NaN loss after first batch	LR too high at step 0; gradient explodes	Add warmup; add gradient clipping
<code>padding_idx</code> tokens affecting loss	Padding tokens included in loss	Use <code>ignore_index=padding_idx</code> in the loss
Encoder-decoder mismatch error	d_{model} differs between encoder and decoder	Ensure both use the same d_{model} ; cross-attention projections require matching dimensions